

Summary of “Parrot: Efficient Serving of LLM-based Applications with Semantic Variable”

Krithika Venkatasubramanian (krithiv), Kristine McLaughlin (ksmcl), Thomas He (mperform)

Problem and Motivation

LLM-based applications are increasing in popularity, from AI coding agents like Cursor and Github Copilot to AI tutors like Khan Academy’s Khanmigo and beyond. These applications work by making multiple sequential or parallel calls to LLM endpoints and collating these outputs for a final response to the user. The authors refer to these LLM APIs as “request-centric”: API calls are treated as independent requests with no representation of relationships between requests. There are three inefficiencies the authors identify with this approach:

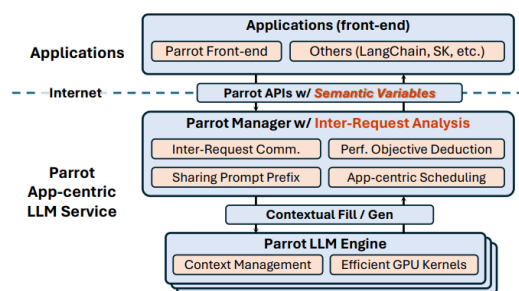
- These APIs lack knowledge of related requests, which could be processed consecutively to reduce network latency and queuing delays.
- Depending on the task, application users might prefer to optimize for throughput over latency, but there is no way for the user to express this. Thus, LLM services always optimize for latency.
- Often, LLM applications exhibit high degrees of redundancy in prompts of their requests such as long system prompts that define the LLM model behavior.

Related Works

The authors mention three related systems that aim to address the same inefficiencies as Parrot does:

- **Deep Learning Serving Systems:** The authors mention tools like Clipper, AlpaServe, and TensorFlow Serving, and their techniques such as caching, scheduling, and model placement. However, these techniques are mainly used for serving deep learning models, which are different in architecture and requirements to LLMs. The authors describe these as “orthogonal” to Parrot.
- **LLM Orchestrator Framework:** The authors refer to tools like LangChain, which help application developers easily organize their models. These tools are helpful for applications that use multiple LLMs in a chain to generate responses. Again, the authors describe these systems as “orthogonal” to Parrot as they focus on how to route single requests through multiple LLMs, rather than focusing on cross-request optimization.
- **DAG-aware System Optimization:** The authors describe two approaches to this kind of optimization. The first approach, employed by tools like Graphene, uses DAGs to represent dependencies between tasks in data analytics workloads, which helps find an optimal schedule for these tasks. The second approach, employed by tools like SONIC, uses information about cost, latency, and communication to increase the efficiency of serverless computing tasks. Parrot extends these tools to LLMs and standardizes how dependencies are exposed to LLMs using Semantic Variable rather than a graph.

Solution Overview



(Figure 6 from the paper)

Parrot provides a Semantic Variable (SV) abstraction, which reveals how LLM requests connect via dataflow analysis. The system can then use this information to more optimally schedule and deduplicate shared work. The Semantic Variable API allows developers to provide annotations for and label “semantic roles” within prompts, with the goal of whether throughput or latency is more important for their tasks. These SV annotations are used by the Parrot Manager for two different analysis types:

- **DAG-based analysis:** the Parrot Manager allocates a session for each user. As requests come in, the Parrot Manager then uses the Semantic Variable annotations to build a DAG, with nodes representing requests and edges representing connections between Semantic Variables in discrete requests. The Parrot Manager can then parse the DAG to find the optimal scheduling for requests.
- **Prompt-structure analysis:** Parrot Manager is able to use prefix hashing to quickly find prompt prefixes, which is an easy way to identify requests that come from the same application. This can then be used for prompt deduplication.

Using SVs and Prompt Structure Analysis, the authors provide 4 optimizations for LLM applications:

- **Serving dependent requests:** the Parrot Manager uses a graph algorithm to traverse the DAG provided in an efficient order. It uses message queues to pipeline information from one request to another, and has message transformation capabilities should this information need to be altered across requests.
- **Performance Objective Deduction:** The Parrot Manager takes into account the user’s throughput/latency annotations when scheduling tasks. It prioritizes the latency of the entire DAG over the latency of a single request.
- **Sharing Prompt Prefix:** The Parrot Manager stores prefix hashes in a table for fast lookup. The authors also propose a more efficient attention decoding algorithm that differentiates between shared and unique tokens, which creates speedups when multiple requests with the same prefix come in.
- **Application-Centric Scheduling:** The Parrot Manager tries to schedule tasks across in GPUs such that latency tasks are scheduled on different GPUs than throughput tasks. Throughput and latency have opposing goals, so this separation helps improve efficiency. Additionally, the Parrot Manager tries to schedule related requests (for example, those which share the same prompt prefixes) on the same GPUs to take advantage of Parrot’s context fork. It does so via a scheduling algorithm that the authors discuss in the paper.

Limitations

Though the authors did not discuss much about Parrot's limitations, we have identified a few from the presentation and class discussion. To begin with, Parrot requires a lot of human intervention to work. It would not work out-of-the-box if an application is using a cluster belonging to OpenAI or Google. Rather, the application developers would have to host their own cluster and implement Parrot in said cluster. Additionally, there is a lot of overhead for application developers to provide these annotations; they have to annotate every request functionality by hand, effectively building the DAG themselves. There is no room for these annotations to be changed dynamically or for them to be inferred without a human in the loop. Finally, it is possible for the application developers to perform metric gaming, artificially inserting dependencies where there aren't any to force Parrot to preferentially schedule their requests. Parrot currently does not do anything to mitigate this.

Future Research Directions

One major future direction for research proposed by the authors is scheduling fairness. This is very analogous to discourse surrounding operating system scheduling of threads: when there is a metric by which LLMs decide which request to process next (or, relatedly, for OSes to decide which threads to schedule next), applications are then motivated to hack this metric and force LLMs to prioritize their requests. Preventing this in operating systems has been a major area of research for OS researchers, and is an area of interest that branches off this paper as well.

Summary of Class Discussion

Q: First graph: As the output length increases, does Parrot end up losing out to vLLM?

A: No, the returns diminish, but Parrot still has the advantage over vLLM. Generally, speedups are because of fewer round trips between the clients and servers, but as output lengths grow larger generation dominates, so the speedups are less. It's also noted that the output length tokens on the graph for what was tested is quite small – 100 tokens in a realistic setting is not that many.

Q: How generalizable is Parrot to other programs with multi-modal inputs?

A: Images might still have similar prefixes and these queries generally still have the same format. A ViT can be added to encode the image, so it likely wouldn't be too difficult to extend.

Q: For this to work, will the LLM providers (like OpenAI or Gemini) have to implement Parrot?

A: Yes, so you cannot use OpenAI or Gemini as is, you would have to host your own cluster and then implement Parrot. There is a hope that eventually things will be standardized (for example MCP protocols), so it's possible that one day there will be a standardized implementation for these annotations. More APIs are starting to be available for example OpenAI has multi-task requests APIs to offer parallel task serving instead of just the chat API.

Q: Do these strategies work using different LLMs within a same application?

A: It should work, as long as Parrot is implemented on all these LLMs. It works because the application developer does all these annotations manually, so the DAG is built manually. The DAG for unstructured text would be challenging to build as compared to database querying tasks.

Summary of “Towards End-to-End Optimization of LLM-based Applications with Ayo”

Krithika Venkatasubramanian (krithiv), Kristine McLaughlin (ksmcl), Thomas He (mperform)

Problem and Motivation

In recent years, there has been an exponential growth in the popularity of LLM-based applications. Most optimization efforts have been focused on the LLM inference latency, as that is one of the LLM’s most important features for these applications and a natural area of interest for optimization. However, when looking at end-to-end performance of these applications, there is still high latency and thus room for improvement at the application level, in the steps these applications have introduced outside of the prompt’s lifecycle in the actual model. In current LLM-based application frameworks, the components can be divided into LLM and non-LLM. In many workflows, non-LLM components significantly contribute to the end-to-end latency, making this problem important to investigate.

Related Works

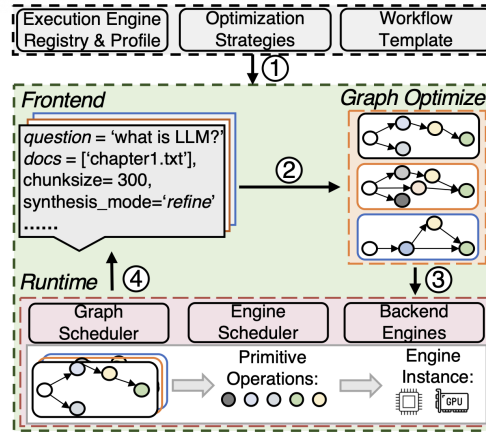
LangChain and LlamaIndex are two popular frameworks that build end-to-end pipelines for applications with multiple task modules. To get a better idea of what modules are, some examples are searching, embedding, indexing, API calls, and LLM decoding.

- **LangChain:** This framework became popular because it enabled developers to connect components like LLMs, databases, and APIs to more easily build these LLM-based systems. It structures applications as chains, where each component in the chain is a module.
- **LlamaIndex:** This framework focuses on a popular LLM application technique, retrieval-augmented generation (RAG). It makes it easier to connect LLMs with knowledge sources.

Both frameworks run modules independently and sequentially, even when they could be executed in parallel. Modules are treated like black boxes and the internal functionality is not considered. Finally, scheduling is done at the request-level, meaning it does not consider workflow dependencies or shared resources. Since they don’t have any application-level information, the room for optimization is limited to per-request latency. Overall, these orchestration tools are **coarse-grained** and **module-based**. These factors lead to serious inefficiencies, with some cases involving non-LLM components being responsible for over half of the total end-to-end latency.

Solution Overview

The key insight that powers the authors’ solution centers around making this framework finer-grained, with task primitives as the scheduling units, instead of optimizing each module or request independently. Task primitives are the smallest operations, such as embedding creation, prefilling, decoding, or ranking. This allows you to represent the application as a primitive-level dataflow graph, enabling parallelization and pipelining across modules. In addition, this dataflow graph includes request correlation and dependency information that enables application-aware scheduling, bridging the gap between the request-level optimization and the application level performance needs.



(Figure 5 from paper)

Ayo's architecture can be broken down into offline stage and online stage. During the offline stage, the developers configure the types of workflows and operations they plan on using for serving. The online stage is triggered when it receives a query with the workflow configurations. Based on this, it creates the primitive-based dataflow graph and applies optimization strategies to generate the execution graph. This execution graph is submitted to the runtime scheduler, who batches the tasks to the execution engines.

Offline

- **Execution Engine:** Provided and configured by the developers. They handle the various requests from the workflow components. There are two types of engines: model-based and model-free. The developers must specify the engine's category, latency profiles, and required resources. Latency profile can be its batch size and sequence length. The required resources specifies the GPU and CPU resources the engine uses.
- **Workflow Template:** Defined by the developers in a high-level. The developers specify the execution engines, their use case, and their I/O configurations. In addition, developers can also provide optimization hints to facilitate the graph optimization stage.

Online

- **Primitive-based Dataflow Graph (p-graph):** Upon receiving the request with the input data and workflow configuration, the Graph Optimizer creates a more granular p-graph using its graph transformation algorithm.
- **Optimization:** The optimization goal is to maximize the parallelism in distributed execution by exploiting primitives parallelization and pipeline parallelism. This is achieved by identifying the unnecessary dependencies throughout the workflow and creating independent primitives, thus allowing them to be executed in parallel. Additional optimizations on LLM workflow have also been introduced such as causal prefilling and streaming decoding output to maximize concurrency with other components in the workflow.
- **Runtime Scheduler:** Ayo adopts a two-tier scheduling architecture – Graph Scheduler and Engine Scheduler. The Graph Scheduler is responsible for managing the execution flow of the optimized execution graph. The Engine Scheduler is responsible for managing the execution engines and the physical resources.

Limitations

The authors discuss a few limitations of Ayo:

- Ayo is designed for the common scenario of LLM-based workflows being manually designed by humans. However, recently, LLMs are starting to plan and execute actions without a fixed workflow, which is called dynamic workflows. The authors acknowledge Ayo would not perform well with dynamic workflows because Ayo performs graph optimizations ahead-of-time instead of just-in-time. In these dynamic cases, it wouldn't be able to capture the complete primitive-level dataflow graph before execution.
- Ayo is a little more invasive than most frameworks, as it requires changes to be made to the engines for some of its optimizations to work. This is what enables finer-grained optimization, but also causes increased overhead for developers.
- Ayo doesn't evaluate which nodes are critical bottlenecks, directly related to the concept of critical paths: the longest chain of dependent operations that determines how fast the entire workflow can finish. This leaves a space for even further optimizations, if less critical nodes are being computed before higher priority ones.

Future Research Directions

Expanding on a couple of the limitations, the authors state that adapting Ayo to dynamic frameworks would be a promising area of future research as these types of workloads rise in popularity. Additionally, further research on the critical path problem using critical path analysis could be used to improve resource allocation between critical and non-critical paths. This could also benefit request scheduling if prioritizing critical nodes for certain queries could improve the topological batching strategy.

Summary of Class Discussion

Q: Why did they choose the primitives they did? Not something more granular?

A: After registering the node, Ayo will decompose the node into more fine-grained primitives that then become primitives, which will be optimized. This decomposition is intuitive and compatible with what already works well with LLM pipelining. Too fine-grained primitives would introduce a lot of overhead and make scheduling extremely difficult or just infeasible.

Q: Is Ayo generalizable to multimodal inputs?

A: Prefix hashing can still happen, even with things like images. Adding multimodality does not dramatically change the workflow, so many of Ayo's optimizations should still be applicable.

Q: Are the LLMs being used IRL publicly accessible? If they were private within their own company and didn't share their LLM with others, wouldn't this be less relevant?

A: In theory you can use any LLM, and the LLM can be in any cluster. These papers assume that where the application is running and where the LLM are running are far apart, so there is a lot of interaction and network round trip. This is a reasonable assumption for some cases, not so much in others, it depends on the application and the way that it's implemented. A lot of popular LLMs that companies would use for their agentic systems are from large companies like OpenAI that offer LLM as a service and are accessible via API, so this is a realistic issue.